Action Success Is Not Task Success: Deterministic Robot-Profile Binding and Replayable Contract Monitoring

Anonymous Submission

Abstract

Robot actions can terminate successfully without establishing the task condition that motivated them. We introduce an auditable deployment boundary that binds an environment-specific task to exactly one compatible robot-profile implementation and evaluates its operationally supported grounded conditions using separate executor-status and predicate events. Capability-parameterized task contracts separate ordered task steps, capability clauses, implementation requirements, and robot-profile facts. Our exactly-one binder rejects missing, ambiguous, and morphology-incompatible realizations and emits normalized diagnostics or a canonical binding record. Runtime monitoring consumes a sequence-ordered, Robot Operating System (ROS)-message-independent contract trace; an `ACTION_STATUS=SUCCESS` event determines when a postcondition is checked but does not establish the postcondition's truth. Across all four permutations of the two tested implementation-list fields, the accepted fixture produced byte-identical canonical binding records and the ambiguous fixture produced byte-identical diagnostic payloads, while source-byte provenance hashes differed as expected. Six nominal controlled traces remained conformant. Across six real evidence-suppression executions and twelve offline trace mutations, all 18 non-nominal cases produced the expected first direct violation, and the observed joint outcome of the three evaluators matched the prespecified tuple in 24/24 cases. Streaming and replay agreed in 12/12 controlled cases and, after deterministic migration, in 38/38 replay-eligible route-complete Benchmark for Autonomous Robot Navigation (BARN) cases. In a bounded two-world BARN study, 38/40 scenarios completed with their expected outcome. A missing postcondition means task success is unestablished; it does not necessarily prove that the physical effect is false.

Introduction

Robot application developers often need a task artifact to survive changes in the mobile base beneath it. A differential-drive base and an Ackermann base can both navigate to two regions, inspect each region, and return to a final location, but they cannot realize every local maneuver in the same way. The differential robot can rotate in place; the Ackermann robot must maintain a positive turning radius. If those facts appear only in launch files, controller parameters, or skill implementations, the task artifact no longer states what semantic obligations are being carried across robots. If, instead, the task directly names a concrete maneuver such as a spin scan, it rejects a robot that could satisfy the same task-level inspection effect by following an arc.

We propose capability-parameterized task contracts. A task is an ordered sequence of abstract capability invocations plus invariants and goals. A capability contract declares typed inputs, preconditions, hold conditions, observable postconditions, failure outcomes, resource and duration bounds, and alternative implementations with requirements. A robot profile declares morphology, kinematic properties, sensors, resources, and the implementations it exposes. Binding either selects exactly one implementation per step and emits a canonical binding record, or fails with stable diagnostics.

This paper studies an auditable deployment boundary in which an abstract task must bind to exactly one compatible implementation using explicit robot-profile facts. Executor status and predicate evidence are semantically distinct: an `ACTION_STATUS=SUCCESS` event determines when a postcondition is evaluated, but the postcondition's truth is determined from predicate state. A ROS-message-schema-independent trace therefore records executor-status and predicate-source events separately. In ROS-style systems (Quigley et al. 2009; Macenski et al. 2022), an action result establishes the action server's own success criterion. Unless that criterion is explicitly mapped to the task contract, it need not establish the task-level predicate or tolerance. Thus, navigation SUCCESS does not by itself prove `at(target=A)` under the task tolerance, and inspection SUCCESS does not by itself prove `observed(target=A)`.

Objective. We evaluate whether a fixed environment-specific task can be deterministically bound to one morphology-compatible implementation per step and monitored during execution and replay without equating executor success with task-level predicate satisfaction.

The evaluation addresses three research questions:

- **RQ1 – Cross-morphology reuse:** Within each tested environment, can the same task artifact be bound unchanged to differential-drive and Ackermann profiles while selecting exactly one compatible implementation per step?

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

- **RQ2 – Compatibility and determinism:** For the tested missing, incompatible, ambiguous, and preference-resolved fixtures, does binding produce the prespecified diagnostic or canonical record, and are normalized outputs invariant under the four tested declaration-order combinations?
- **RQ3 – Runtime semantics and replay:** For nominal, postcondition-not-established, transient-hold, and task-relative-deadline traces, which obligations are distinguished by status-only, success-time-postcondition, and version-0.1 contract evaluation, and do streaming and replay produce identical normalized reports?

The contributions are:

- a versioned representation and profile-sensitive exactly-one binder with stable accepted or diagnostic records;
- a bound-task event semantics that separates executor status from predicate evidence and is shared by streaming and replay; and
- a bounded conformance evaluation across two mobile-base kinematic classes, controlled semantic fixtures, and two prequalified BARN worlds.

We do not claim universal morphology portability, arbitrary-program verification, Nav2 correctness (Macenski et al. 2020), hardware transfer, perception robustness, or broad BARN coverage (Perille et al. 2020). The contribution is the integrated, auditable deployment boundary rather than a new planner, perception method, or navigation algorithm.

Related Work

Planning and execution frameworks. ROS and ROS 2 provide the middleware substrate for many robot systems (Quigley et al. 2009; Macenski et al. 2022). ROSPlan embeds task planning in ROS systems, while PlanSys2 provides a ROS 2 planning framework with PDDL-oriented task planning and behavior-tree execution of plans (Cashmore et al. 2015; Martín et al. 2021). These systems address richer planning and dispatch problems than those addressed in this paper. Our artifact studies a narrower exact-one deployment boundary: a fixed ordered task is bound to one compatible implementation per step, and the resulting obligations are checked from serialized traces.

Skills, capabilities, and task models. Skill-based programming separates reusable robot abilities from task-level composition. Capability and skill surveys emphasize explicit task/skill/primitive distinctions (Pantano, Eiband, and Lee 2022), while SkiROS2 combines world modeling, planning, reactive execution, and skills with pre-, hold-, and postconditions (Mayr, Rovida, and Krueger 2023). Capability ontologies can infer capabilities and affordances from robot components and lower-level abilities (Dussard, Sarthou, and Clodic 2023). Our representation is deliberately non-ontological and operational: it stores explicit robot-profile facts separately from capability contracts, makes one auditable exactly-one deployment choice, and uses the resulting obligations in a shared streaming/replay trace semantics.

Behavior trees. Behavior trees are widely used for modular robot control and are central to many practical navigation systems (Colledanchise and Ögren 2018). Nav2 itself uses behavior trees for navigation task execution (Macenski et al. 2020, 2023). A manually instrumented behavior tree can include condition nodes that detect a missing postcondition; the policy is not prescribed by the behavior-tree formulation alone. This paper stores postcondition and hold obligations as reusable capability data and evaluates their truth from predicate state rather than inferring it from the executor's terminal status.

Formal specifications and runtime verification. Temporal logic has been used to specify and synthesize robot missions (Kress-Gazit, Fainekos, and Pappas 2009), and runtime verification provides general techniques for checking executions against formal specifications (Leucker and Schallhart 2009). Contract-based mission frameworks refine or repair formal mission specifications against libraries of preimplemented specifications and compose context-specific sub-missions with synthesized controllers (Mallozzi et al. 2023a,b). Robotics-specific systems such as ROSRV and ROSMonitoring monitor ROS messages or robot executions against temporal properties (Huang et al. 2014; Ferrando et al. 2020); ROSMonitoring 2.0 extends interface-level monitoring to services and ordered topics (Ghaffari Saadat et al. 2024). CONVINCE studies a broader verifiable robotics toolchain for deliberation systems (Street et al. 2024).

Our problem is narrower: task order and implementation alternatives are authored in advance; profile-sensitive binding selects one implementation and produces obligations for a bound-task-level event trace. The artifact neither repairs the mission nor synthesizes a controller, and it does not interpose on arbitrary ROS interfaces.

Navigation benchmarks and morphology. BARN provides static navigation environments for benchmarking metric ground navigation in cluttered spaces (Perille et al. 2020). We use static BARN worlds as a bounded supporting execution study for the contract harness, not as evidence of broad navigation coverage. The final BARN evaluation uses two runtime-qualified static worlds, and that environmental breadth limitation is reported explicitly.

Representation

The artifact contains three YAML artifact types: task specifications, capability contracts, and robot profiles. Each artifact has schema version 0.1, unknown fields are rejected, numeric values must be finite, and YAML data is never evaluated as code. The representation is intentionally small: tasks are sequential, predicates are ground comparisons over literals, and binding chooses among authored implementations rather than planning new ones.

Tasks. We write a task as

$$T = \langle S, \Phi, G \rangle,$$

where $S = [s_1, \ldots, s_n]$ is an ordered sequence of abstract capability invocations, Φ contains global invariants and temporal constraints, and G contains task-level goal conditions. A step contains a step identifier, an abstract capability identifier, and literal arguments. The controlled task uses the route HOME $\rightarrow A \rightarrow B \rightarrow$ HOME with inspection at both A and

B. The BARN task uses the same five-stage logical structure but ends at FINISH: START $\rightarrow A \rightarrow B \rightarrow$ FINISH. The controlled and BARN YAML files differ because their final-location semantics differ; within each environment, the same task artifact is shared across the two morphologies.

Capability contracts. Capability contracts. A capability contract is

$$C = <I, P, H, Q, R, U, D, F>,$$

where I contains typed inputs, P preconditions, H hold conditions, Q observable postconditions, R implementation-compatibility and required-recovery clauses, U capability- and implementation-level resource declarations, D a capability-duration upper bound, and F a declared failure-outcome vocabulary. Conditions use the operators `eq`, `ne`, `lt`, `le`, `gt`, and `ge`. A placeholder such as `$target` is resolved only against step arguments. Table 1 summarizes the evaluated inspection contract. The complete YAML artifact is provided in the anonymous supplementary material.

In schema version 0.1, the binder enforces Boolean, numeric, sensor, and required-recovery clauses. Resource declarations U are serialized but are not enforced. The duration bound D contributes to a conservative summed-duration check against the task-level deadline; the task-level deadline itself is checked from event timestamps at runtime. Declared failure outcomes F are parsed as part of the capability schema but are not propagated into the bound task or interpreted by the runtime monitor in this version.

The evaluated `observed(target)` predicate denotes geometric inspection coverage: while an inspection step is active, the robot remains within the semantic target's range and field of view for the dwell time. It is not a camera-recognition predicate in this artifact.

Robot profiles. A robot profile declares morphology, Boolean properties, numeric kinematic properties, available sensors, resources, and implementation identifiers. The differential-drive profile declares `can_turn_in_place=true`, minimum turning radius 0, and implementations `nav2_diff` and `spin_scan`. The Ackermann profile declares `can_turn_in_place=false`, a positive minimum turning radius, and implementations `nav2_hybrid` and `arc_scan`. Ackermann turning radius is checked against $R_{\min} = \text{wheelbase}/\tan(\text{steering_limit})$ within an explicit tolerance. The resulting morphology-specific selections and the forced Ackermann rejection are summarized in Table 2.

Trace records. Execution records a ROS-message-schema-independent contract trace. Listing 1 shows the two event types needed to illustrate the ordering rule: predicate evidence is recorded before the corresponding action-success event.

```
{"seq": 12,
 "event_type": "PREDICATE_UPDATE",
 "predicate_updates": [{
   "predicate": "observed",
   "arguments": {"target": "A"},
   "value": true
 }]}
{"seq": 15,
 "event_type": "ACTION_STATUS",
 "step_id": "inspect_a",
 "implementation_id": "spin_scan",
 "action_status": "SUCCESS"}
```

Listing 1: Predicate evidence preceding the corresponding action-success event in sequence order.

The `seq` field is strictly increasing within one trace. Timestamps must be finite and nondecreasing; equal timestamps are resolved by `seq`. Replay consumes events in serialized sequence order and does not sort by timestamp. Predicate updates persist in monitor state until a later update changes them. A postcondition is evaluated when its `ACTION_STATUS=SUCCESS` event is processed, using the current predicate state after all earlier-in-sequence predicate updates have been applied. Version 0.1 uses state-based postconditions and does not require the supporting predicate update to occur after the current step's `STEP_STARTED` event. Later evidence cannot retroactively erase an already emitted violation.

Binding and Monitoring

Figure 1 summarizes how authored artifacts produce a bound task, separate execution evidence, and matched streaming and replay reports.

Exact-one binding. For each task step, the binder looks up the abstract capability, checks required arguments and literal types, enumerates robot-exposed implementation candidates, evaluates implementation requirements, and grounds the step's preconditions, hold conditions, and postconditions. If no implementation is compatible, binding fails. If multiple implementations are compatible and no explicit preference disambiguates them, binding also fails. The tested capability-contract and robot-profile implementation-list orders are normalized before reporting; their four permutations therefore produced the same normalized output in the evaluated fixtures.

Let $c(s)$ denote the capability invoked by task step s, let $I_{c(s)}$ be the implementation identifiers declared by its capability contract, and let $E_p(c(s))$ be the identifiers exposed by robot profile p. After argument and profile-consistency validation, the compatible implementation set is

$$K(s,p) = \left\{ i \in I_{c(s)} \cap E_p(c(s)) \;\middle|\; \text{Req}(i,p) \right\},$$

where $\text{Req}(i,p)$ holds when the implementation's Boolean, numeric, sensor and required-recovery requirements are satisfied by p. If $E_p(c(s)) = \varnothing$, or if the profile exposes an identifier absent from $I_{c(s)}$, the binder emits `E_MISSING_IMPLEMENTATION`. If $K(s,p) = \varnothing$, it returns the accumulated requirement diagnostics. If a profile preference is declared and belongs to $K(s,p)$, the preferred implementation is selected. If a preference is declared but does not belong to $K(s,p)$, the binder emits `E_MISSING_IMPLEMENTATION`. If no preference is declared, the sole member is selected when $|K(s,p)| = 1$, while $|K(s,p)| > 1$ produces `E_AMBIGUOUS_IMPLEMENTATION`.

Clause	Evaluated condition	Checked at	Evidence/source
Precondition	`at($target) = true`	`STEP_STARTED`	Current predicate state
Hold condition	`outside_restricted_zone = true`	`STEP_STARTED` and later events while active	Predicate state
Hold condition	`localization_confidence` ≥ 0.75	`STEP_STARTED` and later events while active	Localization source
Postcondition	`observed($target) = true`	`ACTION_STATUS=SUCCESS`	Current predicate state at SUCCESS
Duration bound	15.0 s	Binding-time summed-duration check	Capability contract
Sensor requirement	`lidar` present	Binding	Robot-profile sensor list
Required recovery	`recover` implementation exposed	Binding	Robot-profile implementation map
`spin_scan` requirement	`can_turn_in_place = true`	Binding	Robot-profile Boolean field
`arc_scan` requirement	`available_scan_radius_m` $\geq$ `minimum_turning_radius_m`	Binding	Robot-profile numeric fields

Table 1: Evaluated inspection contract and enforcement phase. Resource declarations and failure-outcome labels are represented but are not operationally enforced in version 0.1.

Profile	can_turn_in_place	$R_{\min}$ (m)	Navigate	Inspect
Differential	true	0.00	`nav2_diff`	`spin_scan`
Ackermann	false	0.57	`nav2_hybrid`	`arc_scan`
Ackermann + forced spin_scan	false	0.57	`nav2_hybrid`	`E_REQUIREMENT_BOOLEAN`

Table 2: Morphology-specific binding. A forced Ackermann `spin_scan` binding is rejected with `E_REQUIREMENT_BOOLEAN`.

The inspection example shows the mechanism. The differential profile satisfies the Boolean requirement for `spin_scan`. The Ackermann profile does not, so a forced Ackermann `spin_scan` binding fails with static diagnostic `E_REQUIREMENT_BOOLEAN`. The Ackermann profile instead satisfies the numeric scan-radius requirement for `arc_scan`. The task still names only `inspect(target)`.

Provenance manifests and canonical binding records. Each binding attempt stores a deterministic provenance manifest. A successful binding additionally stores a deterministic canonical binding record; a failed binding stores a normalized static diagnostic. The provenance manifest records artifact-relative paths plus `source_sha256` over original input bytes and `canonical_sha256` over normalized semantic content. The canonical binding record contains only design-time binding information: normalized task, capability-library, and robot-profile hashes; ordered bound steps; every candidate implementation for every step; candidate compatibility and stable rejection codes; explicit preferences; selected implementations; grounded preconditions, hold conditions, and postconditions; and represented duration and resource bounds. It contains no original source-byte hash, wall-clock timestamp, absolute path, ROS message, semantic-world parameter, or predicate-source configuration. Consequently, the record supports auditing of the binding decision but is not, by itself, a complete runtime re-execution record.

Static diagnostics use the `E_`$\star$ namespace. The implemented suite covers a missing implementation selection (`E_MISSING_IMPLEMENTATION`), ambiguous implementation (`E_AMBIGUOUS_IMPLEMENTATION`), forced Ackermann spin scan (`E_REQUIREMENT_BOOLEAN`), inconsistent Ackermann kinematics (`E_PROFILE_KINEMATICS`), a deadline not guaranteed by duration upper bounds (`E_DEADLINE_NOT_GUARANTEED`), explicit preference resolution, and declaration-order determinism. These checks reject represented deployment incompatibilities; they are not a proof of arbitrary robot-program correctness.

Monitor semantics. The runtime monitor processes trace events in serialized `seq` order. Sequence numbers must strictly increase, timestamps must be nondecreasing, and equal timestamps are resolved by `seq`. Legacy traces without `seq` are migrated deterministically from serialized list order; replay does not resort events by timestamp.

For each event, predicate updates are first applied to the monitor state and the event transition is then processed. Preconditions are evaluated at `STEP_STARTED`. If a step remains active after the transition, its hold conditions are evaluated; global invariants and the task deadline are evaluated after every event. Postconditions are evaluated from current predicate state when the corresponding `ACTION_STATUS=SUCCESS` event is processed; version 0.1 applies no action-instance-freshness constraint, and terminal goals are evaluated at `TASK_FINISHED` or replay finalization. Required conditions must be positively established: missing values and explicit nulls become UNKNOWN, and UNKNOWN does not satisfy a required condition.

Later predicate events do not retroactively remove an emitted violation. Runtime deadline checking is event-driven: `V_DEADLINE_EXCEEDED` is emitted at the first processed event whose timestamp exceeds the task deadline. Hold-condition and invariant guarantees are event-sampled; a violation that occurs and recovers entirely between recorded events may be missed.

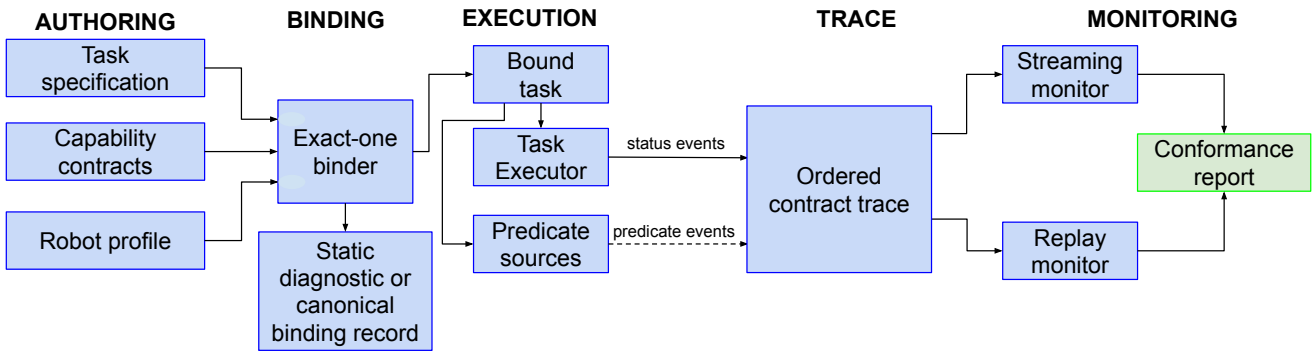


Figure 1: System data flow. Task, capability, and robot-profile artifacts feed an exactly-one binder. Successful binding produces a bound task and canonical binding record, whereas unsuccessful binding produces a static diagnostic. The bound task configures the executor and predicate sources. The executor emits status events, while the predicate sources emit separate predicate events; both enter the same ordered ROS-message-schema-independent contract trace, which is interpreted by streaming and replay monitors.

The key semantic rule is

$$\texttt{ACTION_STATUS=SUCCESS} \not\Rightarrow Q.$$

Postconditions Q must be established by predicate observations in the trace. If a skill reports SUCCESS but the corresponding grounded postcondition is false or unknown, the monitor emits runtime violation `V_POSTCONDITION_NOT_ESTABLISHED`. Runtime violations use the `V_*` namespace, distinct from static diagnostics.

Reference evaluators and semantic ablations. The status-only ablation accepts only when every required task step terminates with SUCCESS; it ignores predicates, hold history, invariants, deadlines, and goals. The success-time-postcondition reference evaluator also requires every step to terminate with SUCCESS and checks the current grounded postconditions at each successful completion. It ignores historical hold violations, global invariant history, task deadlines, and final goals beyond per-step postconditions. The full contract monitor is the only evaluator in the paper that checks the represented history-sensitive hold and deadline clauses.

These evaluators isolate the semantic contribution of each contract layer; they are not competitive implementations of existing planning, skill, behavior-tree, or runtime-verification frameworks.

Direct and derived violations. Reports classify runtime violations by causal role. A direct violation is the condition that defines the failure; a derived violation is a later consequence. In the evidence-suppression scenario, `inspect(A)` returns SUCCESS while `observed(A)` is not established. The direct violation is `V_POSTCONDITION_NOT_ESTABLISHED`. If the final goal also lacks `observed(A)`, `V_GOAL_NOT_ESTABLISHED` is reported as a derived terminal consequence.

Implementation

The prototype is implemented as three layers. The core task-contract package has no ROS, Gazebo, Nav2, or rosbag dependency. It owns YAML loading, dataclasses, binding, static validation, normalized binding records, trace representation, status-only and success-time postcondition evaluators, semantic monitoring, deterministic mutations, reports, and a CLI.

The ROS adapter depends on the core package and translates ROS execution into the trace model. Navigation uses a direct `rclpy.action.ActionClient` for `nav2_msgs/action/NavigateToPose`. Goal acceptance maps to RUNNING; successful action results map to SUCCESS; canceled results map to CANCELED; rejected, aborted, or unknown results map to FAILURE. Feedback is stored only as bounded scalar summaries. The adapter never uses navigation success to establish `at(target)`.

Inspection implementations publish velocity commands. `spin_scan` uses zero linear velocity and bounded angular velocity, then publishes a final zero twist. `arc_scan` uses positive linear velocity and bounded angular velocity, validates curvature against the Ackermann profile, rejects zero-linear nonzero-angular commands, and also publishes a final zero twist. The evidence-suppression scenario suppresses the predicate event that would establish `observed(target=A)` while preserving the inspection action status. It does not modify robot motion or assert that a physical visual recognition event was false. Figure 2 illustrates the strict evidence-before-success rule and the resulting evaluator outcomes.

The semantic-world file supplies named targets, target tolerances, observation range and field-of-view parameters, restricted zones, and the localization confidence source. The `at(target)` predicate is derived from the `map` to base transform; missing or stale transforms produce UNKNOWN, not false. The observation predicate is derived only during an active inspection step after the robot remains within the configured range and horizontal field of view for the dwell time. The evaluated predicate is geometric inspection coverage; camera data is not consumed by the predicate source. Thus the adapter has two separate channels for inspection: the skill action can finish, and the predicate source can separately establish or fail to establish the postcondition. Version 0.1 distinguishes executor-status events from separately emitted predicate events, but it does not bind, authenticate,

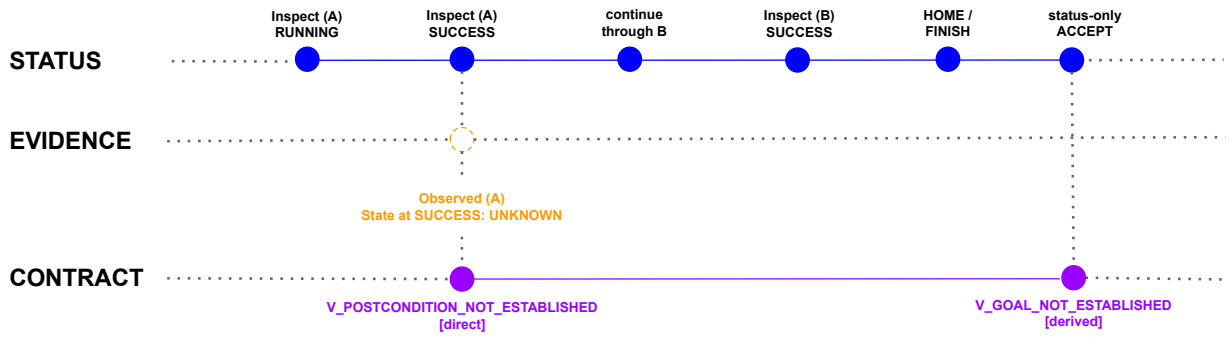


Figure 2: Missing-postcondition-evidence timeline for a controlled execution. When `inspect(A)` reports SUCCESS, no preceding event has established `observed(A)`, so its state is UNKNOWN and the monitor emits direct `V_POSTCONDITION_NOT_ESTABLISHED`. Execution continues through B and returns HOME; task finalization then emits derived `V_GOAL_NOT_ESTABLISHED`. The status-only ablation accepts the action sequence, whereas the success-time-postcondition reference and full contract monitor reject it.

or assess the independence of evidence providers.

The BARN integration is an overlay package. It indexes the pinned static BARN checkout, generates semantic maps from occupancy and reference-path assets, launches the custom robots in selected static worlds, and aggregates qualification and evaluation results. Each run emits a canonical binding record, an ordered trace, evaluator reports, and a provenance manifest containing artifact hashes. Complete runtime re-execution additionally requires the semantic-world artifact and predicate-source configuration.

Evaluation Design

The evaluation combines three evidence sources: real Gazebo/Nav2 executions in a controlled world, real Gazebo/Nav2 executions in a bounded BARN study, and offline replay mutations derived from preserved controlled traces. The offline mutations do not simulate new robot motion; they modify serialized contract traces to exercise monitor clauses under deterministic replay.

This is an oracle-based conformance evaluation of the implemented binding and monitor semantics, supplemented by bounded simulator executions. Prespecified fixture outcomes and mutation oracles assess semantic correctness; the study is not a statistical comparison of navigation algorithms or a test of broad environmental generalization.

Tasks and scenarios. The controlled task executes HOME $\rightarrow$ A, inspects A, navigates to B, inspects B, and returns to HOME. The BARN task executes START $\rightarrow$ A, inspects A, navigates to B, inspects B, and finishes at FINISH. Each task artifact is shared across differential-drive and Ackermann morphologies in its environment.

Each environment has four scenarios: differential nominal, Ackermann nominal, differential evidence suppression, and Ackermann evidence suppression. Nominal scenarios are expected to be action-successful and semantically conformant. In evidence-suppression scenarios, `inspect(A)` reports SUCCESS while the predicate event needed to establish `observed(A)` is suppressed; execution continues through B and the final destination. An evidence-suppression execution produces its prespecified experimental outcome when the status-only evaluator accepts and the full contract monitor emits direct `V_POSTCONDITION_NOT_ESTABLISHED`. This shows unestablished task success, not necessarily a false physical inspection effect.

Controlled and BARN executions. The final controlled evaluation used three repetitions of each of four scenarios, for 12 simulator runs. Ackermann runs were required to have no zero-radius commands and curvature no greater than the profile limit.

The exploratory BARN pilot attempted 24 runs over six offline-selected worlds and was retained as diagnostic evidence. Because offline screening did not guarantee real Nav2 realization, a fixed world qualification procedure was applied to static worlds 0–29 before the final replicate runs. Only worlds 3 and 5 qualified. The final study then used five post-qualification replicate runs per scenario (identifiers 201–205) in these two prequalified worlds, for $2 \times 4 \times 5 = 40$ simulator scenarios.

Binder mutation suite. RQ2 is evaluated by deterministic design-time cases: a missing implementation selection, two compatible implementations without preference, forced Ackermann `spin_scan`, inconsistent Ackermann turning radius, deadline not guaranteed by summed duration upper bounds, explicit preference resolving ambiguity, and all four unique permutations of the capability-contract and robot-profile implementation-list fields in each of two fixtures: an accepted-binding fixture and an ambiguous-binding fixture. The accepted fixture is expected to produce a byte-identical canonical binding record in 4/4 permutations, while the ambiguous fixture is expected to produce a byte-identical normalized diagnostic in 4/4 permutations.

Runtime trace and mutation suite. RQ3 compares three evaluators on six preserved nominal controlled traces and three mutation classes. Missing-postcondition cases use the six preserved simulator evidence-suppression traces. Hold-condition and deadline cases are offline derived from the six nominal traces. The hold mutation sets localization_confidence to 0.0 while inspect(A) is active and restores its original numeric value before completion. The deadline mutation shifts terminal events beyond the represented task deadline. For each derived trace, we record the parent trace hash, mutation parameters, output hash, expected direct violation,

actual first direct violation, status-only result, success-time-postcondition result, and full-contract result.

Metrics. All reported rates include explicit numerators and denominators. Harness completion means that required reports were produced. Nominal mission completion counts nominal runs whose required action sequence completed. Conditional nominal conformance is conditioned on completed nominal runs. An evidence-suppression run is eligible for the missing-postcondition comparison only when `inspect(A)` was reached, reported SUCCESS, and suppression of `observed(A)` was confirmed. Navigation or infrastructure failures before eligibility are not monitor misses.

Results

RQ1: Binding and reuse.

Answer to RQ1: Yes, within the two tested morphologies and evaluated environments, each environment-specific task artifact was reused unchanged while binding to morphology-specific inspection implementations. Across the 12 final-version controlled runs, 12/12 completed the full two-region route and matched their expected outcomes (Table 3). The six nominal controlled runs completed the mission in 6/6 cases and were semantically conformant in 6/6 cases. Differential runs selected `spin_scan`; Ackermann runs selected `arc_scan`. Online and replay monitors agreed in 12/12 controlled runs. Ackermann zero-radius command count was 0; maximum observed curvature was $1.136\,m^{-1}$, below the permitted $1.754\,m^{-1}$.

The bounded BARN study preserved the same binding pattern in runtime-qualified worlds 3 and 5. The final BARN evaluation attempted 40 post-qualification runs in the two prequalified worlds; 38/40 completed the route and matched the expected end-to-end outcome. The two remaining runs failed before mission execution because the Nav2 map server did not become active. Among nominal BARN runs, 19/20 completed the mission; among the 19 completed nominal runs, 19/19 were conformant. Online/replay agreement was 38/38 for route-complete, replay-eligible BARN runs. Ackermann zero-radius command count remained 0 and maximum curvature remained $1.136\,m^{-1}$, below the permitted $1.754\,m^{-1}$. The BARN executions are preserved legacy traces: deterministic `seq` values are assigned by serialized list order during migration, and original online reports and migrated replay reports agree for 38/38 route-complete, replay-eligible BARN runs. Representative routes and morphology-specific inspection motions are shown in Figure 3.

RQ2: Design-time compatibility and determinism.

Answer to RQ2: Yes for the tested fixtures and implementation-list fields. Each incompatibility produced its prespecified diagnostic, explicit preference resolved ambiguity, the accepted-binding fixture produced byte- identical canonical binding records in 4/4 permutations, and the ambiguous-binding fixture produced byte-identical normalized diagnostics in 4/4 permutations.

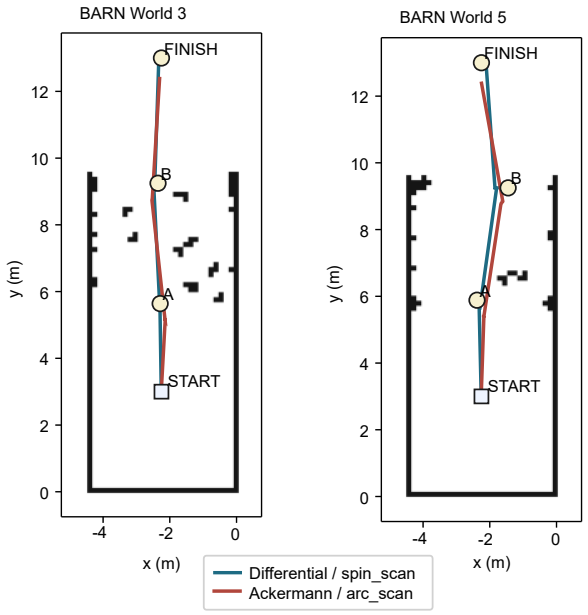


Figure 3: Representative nominal trajectories in BARN Worlds 3 and 5. Both robot profiles execute the semantic sequence START $\rightarrow$ A $\rightarrow$ B $\rightarrow$ FINISH. The differential-drive profile binds inspection to `spin_scan`, producing a heading change with negligible translation, while the Ackermann profile binds inspection to `arc_scan`, producing continuous positive-radius motion. Trajectories are reconstructed from recorded pose and inspection-step events.

Table 5 summarizes the design-time hardening cases. The binder rejected the missing-implementation-selection fixture with `E_MISSING_IMPLEMENTATION`, two compatible implementations without preference with `E_AMBIGUOUS_IMPLEMENTATION`, forced Ackermann `spin_scan` with `E_REQUIREMENT_BOOLEAN`, inconsistent Ackermann radius with `E_PROFILE_KINEMATICS`, and a deadline not guaranteed by summed duration upper bounds with `E_DEADLINE_NOT_GUARANTEED`. An explicit profile preference resolved the otherwise ambiguous case and was recorded in the binding output. Testing all four unique permutations of the capability-contract and robot-profile implementation-list fields in the accepted-binding fixture produced byte-identical canonical binding records in 4/4 cases. Testing the same four permutations in the ambiguous-binding fixture produced byte-identical normalized diagnostic reports in 4/4 cases.

Because the evaluated `observed(target)` predicate uses geometric coverage rather than camera detections, the final capability contract contains no camera requirement.

RQ3: Runtime contract monitoring.

Answer to RQ3: Status-only accepted all action-successful traces; the success-time-postcondition evaluator additionally rejected missing postcondition evidence; only the full monitor rejected transient hold and deadline violations. Streaming and replay agreed on all reported replay-eligible traces.

Setting	Attempted	Harness complete	Route complete	Nominal complete	Conditional nominal conformance	Expected direct-violation match / eligible	Replay agreement
Controlled	12	12/12	12/12	6/6	6/6	6/6	12/12
BARN pilot	24	23/24	–	8/12	7/8	10/10	23/23
Final BARN	40	40/40	38/40	19/20	19/19	19/19	38/38

Table 3: Execution summary. The expected direct violation column is conditioned on eligible evidence-suppression runs; infrastructure or navigation failures before eligibility are not monitor misses.

Case	Status-only	Success-time postcondition	Full contract	Expected full-contract result	Expected tuple matched
Nominal	accept	accept	conformant	no violation	6/6
Missing postcondition evidence	accept	reject	reject	`V_POSTCONDITION_NOT_ESTABLISHED`	6/6
Transient hold	accept	accept	reject	`V_HOLD_CONDITION_FALSE`	6/6
Deadline overrun	accept	accept	reject	`V_DEADLINE_EXCEEDED`	6/6

Table 4: Runtime evaluator outcomes. The nominal row expects conformance; non-nominal rows expect the listed first direct violation. Missing-postcondition-evidence rows use preserved simulator executions; hold and deadline rows use offline trace mutations.

Case	Expected result	Matched / total
Missing implementation selection	`E_MISSING_IMPLEMENTATION`	1/1
Ambiguous impl.	`E_AMBIGUOUS_IMPLEMENTATION`	1/1
Forced Ackermann spin	`E_REQUIREMENT_BOOLEAN`	1/1
Inconsistent Ackermann radius	`E_PROFILE_KINEMATICS`	1/1
Deadline not guaranteed	`E_DEADLINE_NOT_GUARANTEED`	1/1
Explicit preference	accepted, preference recorded	1/1
Accepted-fixture list order	byte-identical canonical binding record	4/4
Ambiguous-fixture list order	byte-identical normalized diagnostic	4/4

Table 5: Design-time binding and diagnostic suite. Camera removal is not included because the evaluated inspection predicate is geometric coverage, not a camera detector.

In controlled evidence-suppression scenarios, 6/6 eligible runs produced direct `V_POSTCONDITION_NOT_ESTABLISHED` while the status-only ablation reported success. The mission continued through B, inspected B, and returned home. In final BARN evidence-suppression scenarios, 19/19 eligible runs produced the same expected direct violation. Derived `V_GOAL_NOT_ESTABLISHED` appeared when the terminal goal still depended on `observed(A)`.

Table 4 compares the evaluators on preserved and offline-derived traces. Status-only accepts all rows because every required action step reports SUCCESS. The success-time-postcondition reference evaluator rejects missing terminal postconditions, but accepts transient hold-condition violations and deadline overruns because it ignores history and deadlines. The full contract monitor rejects each mutated clause with the expected direct `V_*` code. The nominal traces were conformant in 6/6 cases. The 18 non-nominal traces (six preserved evidence-suppression executions and twelve offline mutations) produced the prespecified first direct violation in 18/18 cases. The observed joint outcome of the three evaluators matched the prespecified outcome tuple in 24/24 controlled traces.

Bounded BARN interpretation. The exploratory pilot is reported separately in the artifact. It completed 23/24 harnesses, with 8/12 nominal mission completions, 7/8 conditional nominal conformance, 10/10 eligible missing-evidence matches, and 23/23 replay agreement. Because offline screening did not guarantee Nav2 realization, the final study is interpreted only as a bounded two-world execution study. The two map-server activation failures affect the 38/40 end-to-end rate but not the conditional 19/19 monitor denominators.

Discussion, Limitations, and Conclusion

At the tested scale, one task artifact bound to morphology-specific implementations, incompatible bindings were rejected before execution, and the full monitor rejected unestablished postcondition evidence that status-only evaluation accepted. This is an epistemic claim: a missing predicate event leaves task success unestablished; it does not prove that the physical effect was false. The two BARN map-server activation failures reduce the 38/40 end-to-end rate but do not alter the 19/19 eligible missing-evidence result. The evidence is bounded. The final BARN study used two runtime-qualified worlds; tasks are sequential; predicates are ground comparisons; observation uses geometric coverage without visual recognition or occupancy-grid line-of-sight reasoning; localization confidence is a covariance-derived proxy; and evidence sources are not authenticated. Postcondition checking is state-based rather than action-instance-fresh, and the evaluated tasks do not repeat the same grounded `observed(target)` postcondition; repeated-invocation freshness is therefore outside the evaluated scope. The artifact does not provide arbitrary temporal-logic synthesis, behavior-tree compilation, Nav2 verification, or hardware validation, and hold and invariant guarantees are event-sampled. These restrictions support an auditable conformance study rather than broad claims of portability or environmental generalization. Capability-parameterized task contracts therefore provide a compact deployment boundary between task-level obligations and morphology-specific realization.

References

Cashmore, M.; Fox, M.; Long, D.; Magazzeni, D.; Ridder, B.; Carrera, A.; Palomeras, N.; Hurtos, N.; and Carreras, M. 2015. ROSPlan: Planning in the Robot Operating System. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 25, 333–341.

Colledanchise, M.; and Ögren, P. 2018. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press.

Dussard, B.; Sarthou, G.; and Clodic, A. 2023. Ontological Component-Based Description of Robot Capabilities. In *International Workshop on Working towards Ontology-Based Standards for Robotics and Automation*. ArXiv:2306.07569.

Ferrando, A.; Cardoso, R. C.; Fisher, M.; Ancona, D.; Franceschini, L.; and Mascardi, V. 2020. ROSMonitoring: A Runtime Verification Framework for ROS. In *Towards Autonomous Robotic Systems*, 387–399. Springer International Publishing.

Ghaffari Saadat, M.; Ferrando, A.; Dennis, L. A.; and Fisher, M. 2024. ROSMonitoring 2.0: Extending ROS Runtime Verification to Services and Ordered Topics. In *Proceedings of the Sixth International Workshop on Formal Methods for Autonomous Systems*, volume 411 of *Electronic Proceedings in Theoretical Computer Science*, 38–55.

Huang, J.; Erdogan, C.; Zhang, Y.; Moore, B.; Luo, Q.; Sundaresan, A.; and Rosu, G. 2014. ROSRV: Runtime Verification for Robots. In *Runtime Verification*, 247–254. Springer.

Kress-Gazit, H.; Fainekos, G. E.; and Pappas, G. J. 2009. Temporal-Logic-Based Reactive Mission and Motion Planning. *IEEE Transactions on Robotics*, 25(6): 1370–1381.

Leucker, M.; and Schallhart, C. 2009. A Brief Account of Runtime Verification. *The Journal of Logic and Algebraic Programming*, 78(5): 293–303.

Macenski, S.; Foote, T.; Gerkey, B.; Lalancette, C.; and Woodall, W. 2022. Robot Operating System 2: Design, Architecture, and Uses in the Wild. *Science Robotics*, 7(66): eabm6074.

Macenski, S.; Martin, F.; White, R.; and Ginés Clavero, J. 2020. The Marathon 2: A Navigation System. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Macenski, S.; Moore, T.; Lu, D. V.; Merzlyakov, A.; and Ferguson, M. 2023. From the Desks of ROS Maintainers: A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2. *Robotics and Autonomous Systems*, 168: 104493.

Mallozzi, P.; Incer, I.; Nuzzo, P.; and Sangiovanni-Vincentelli, A. 2023a. Contract-Based Specification Refinement and Repair for Mission Planning. In *2023 IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE)*, 29–38.

Mallozzi, P.; Nuzzo, P.; Piterman, N.; Schneider, G.; and Pelliccione, P. 2023b. Correct-by-Construction Design of Contextual Robotic Missions Using Contracts. *CoRR*, abs/2306.08144.

Martín, F.; Ginés, J.; Matellán, V.; and Rodríguez, F. J. 2021. PlanSys2: A Planning System Framework for ROS2. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Mayr, M.; Rovida, F.; and Krueger, V. 2023. SkiROS2: A Skill-Based Robot Control Platform for ROS. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. ArXiv:2306.17030.

Pantano, M.; Eiband, T.; and Lee, D. 2022. Capability-Based Frameworks for Industrial Robot Skills: A Survey. *arXiv preprint arXiv:2203.00538*.

Perille, D.; Truong, A.; Xiao, X.; and Stone, P. 2020. Benchmarking Metric Ground Navigation. In *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 116–121. IEEE.

Quigley, M.; Conley, K.; Gerkey, B.; Faust, J.; Foote, T.; Leibs, J.; Wheeler, R.; and Ng, A. Y. 2009. ROS: An Open-Source Robot Operating System. In *ICRA Workshop on Open Source Software*.

Street, C.; Warsame, Y.; Mansouri, M.; Klauck, M.; Henkel, C.; Lampacrescia, M.; Palmas, M.; Lange, R.; Ghiorzi, E.; Tacchella, A.; Azrou, R.; Lallement, R.; Morelli, M.; Chen, G. I.; Wallis, D.; Bernagozzi, S.; Rosa, S.; Randazzo, M.; Faraci, S.; and Natale, L. 2024. Towards a Verifiable Toolchain for Robotics. In *Proceedings of the AAAI Symposium Series*, volume 4, 398–403.